

Program Analysis 2025-26

Lecture #22

Control Flow Analysis



UNIVERSITÀ
DI PISA

Cartesian Product Algorithm (CPA)

FUN: syntax

term

$t ::= \dots$

| $\text{fn } x_1, \dots, x_m \Rightarrow e_b$ // function definition
(with m arguments)

$e ::= t^\ell$ | $e_0(e_1, \dots, e_m)$ // function application
(with m arguments)

| $\dots$

- **well-formedness condition**: all function abstractions are closed, i.e.,
 $\text{FV}(\text{fn } x_1, \dots, x_m \Rightarrow e_0) = \emptyset$
- we do not treat recursive functions

The Cartesian Product Algorithm: adding context

We extend the analysis to take **context** into account

In this variant, **contexts** are tuples of argument values

$$\begin{aligned}\delta &\in \Delta = \mathbf{Term}^m \\ \hat{v} &\in \widehat{\mathbf{Val}} = \wp(\mathbf{Term}) \\ \hat{\rho} &\in \widehat{\mathbf{Cenv}} = (\mathbf{Var} \times \Delta) \rightarrow \widehat{\mathbf{Val}} \\ \hat{C} &\in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}\end{aligned}$$

The resulting analysis is a refinement of 0-CFA that partitions the flow of values to expressions and variables according to the actual argument types in their dynamic calling context

Rephrasing the O-CFA analysis: [fn]

$$[fn] (\hat{C}, \hat{\rho}) \models (fn\ x_1, \dots, x_m \Rightarrow e_b)^\ell \text{ iff } \{fn\ x_1, \dots, x_m \Rightarrow e_b\} \subseteq \hat{C}(\ell)$$

Example

$$e \triangleq (fn\ x\ y \Rightarrow (x^1 + y^2)^3)$$

$$\hat{C}(3) \supseteq \{(fn\ x\ y \Rightarrow x^1 + y^2)\} \dots$$

Rephrasing the O-CFA analysis: [app]

$$[app] (\hat{C}, \hat{\rho}) \models (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$$

$$(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models t_m^{\ell_m}$$

$$(\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}) \in \hat{C}(\ell_0) :$$

each argument
flows to its
corresponding
parameter

$$\hat{C}(\ell_1) \times \dots \times \hat{C}(\ell_m) \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge$$

$$(\hat{C}, \hat{\rho}) \models t_b^{\ell_b} \wedge$$

$$\hat{C}(\ell_b) \subseteq \hat{C}(\ell) \wedge \dots$$

In the case where no $\hat{C}(\ell_i)$ is empty
 $\hat{C}(\ell_1) \subseteq \hat{\rho}(x_1) \wedge \dots \wedge \hat{C}(\ell_m) \subseteq \hat{\rho}(x_m)$

Example

$e \triangleq (\text{f}^3(4^4, 6^5))^6$ supposing that $\hat{C}(3) \supseteq \{(\text{fn } x \Rightarrow x^1 + y^2)\}$ then
 $\hat{C}(4) \subseteq \hat{\rho}(1)$ and $\hat{C}(5) \subseteq \hat{\rho}(2)$

Adding context: [var], [fn]

tuple of argument values

$$[var] (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} x^{\ell} \text{ iff } \hat{\rho}(x, \delta) \subseteq \hat{C}(\ell, \delta)$$

$$[fn] (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} (\text{fn } x_1, \dots, x_m \Rightarrow e_b)^{\ell} \text{ iff } \{\text{fn } x_1, \dots, x_m \Rightarrow e_b\} \subseteq \hat{C}(\ell, \delta)$$

Adding context: [app]

$$[app] (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_0^{\ell_0} \wedge$$

we distinguish calls
by what they receive

$$(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_m^{\ell_m} \wedge$$

$$(\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}) \in \hat{C}(\ell_0, \delta) :$$

$$\forall \delta_b \in \hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta) :$$

$$\{\delta_b\} \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta_b} t_b^{\ell_b} \wedge$$

$$\hat{C}(\ell_b, \delta_b) \subseteq \hat{C}(\ell, \delta)) \wedge \dots$$

we analyse the body
separately for each δ_b
no data merging

The Cartesian Product Algorithm derives its name from the cartesian product $\hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta)$ over which δ_b ranges

Observation

- The function body is analysed separately for each possible tuple of arguments
- No merging across different tuples
- This creates redundancy: it leads to many re-analyses of the same body: we need **memoisation** so that the bodies are only analysed once
- We memoise analyses of each $(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_b^{\ell_b}$ as *templates* (with entries like $t_b^{\ell_b}$ associated with $x \mapsto f, y \mapsto a$) and maintain them in a **global pool**
- when a template is created it is only added to the pool if it is not already present (template reuse)

Implementation

- The analysis can be implemented in a straightforward manner because the product grows monotonically:
- $\hat{C}(\ell_1, \delta) \times \cdots \times \hat{C}(\ell_m, \delta)$ will increase each time one of the $\hat{C}(\ell_i, \delta)$ increases (assuming that none is empty)

Exercise

If we remove the **well-formedness condition**, i.e., function abstractions are not closed, i.e., $FV(\text{fn } x_1, \dots, x_m \Rightarrow e_0) \neq \emptyset$, we need to extend the analysis,

- treating functions as closure abstractions:, and
- using new clauses

$$(\hat{C}, \hat{\rho}) \models_{\delta}^{ce} e$$

where

- $\delta \in \Delta = \mathbf{Term}^m$
- $ce \in \mathbf{CEnv} = \mathbf{Var} \rightarrow \Delta$

Adding context: [app]

$$\begin{aligned}
 [app] \quad & (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^{\ell} \text{ iff} \\
 & \text{iff } (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_0^{\ell_0} \wedge (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_m^{\ell_m} \\
 & (\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}) \in \hat{C}(\ell_0, \delta) : \\
 & \quad \forall \delta_b \in \hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta) : \\
 & \quad \quad \{\delta_b\} \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge \\
 & \quad \quad (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta_b} t_b^{\ell_b} \wedge \\
 & \quad \quad \hat{C}(\ell_b, \delta_b) \subseteq \hat{C}(\ell, \delta)) \wedge \dots
 \end{aligned}$$

we analyse the body
for each δ_b

The Cartesian Product Algorithm derives its name from the cartesian product $\hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta)$ over which δ_b ranges

Clause for function application

$$\begin{aligned}
 [app] \ (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } & (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \\
 & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}, ce_0) \in \hat{C}(\ell_1, \delta) : \\
 & (\hat{C}, \hat{\rho}) \models_{\delta_0}^{ce_0} t_0^{\ell_0} \wedge \\
 & \hat{C}(\ell_2, \delta) \subseteq \hat{\rho}(x, \delta) \wedge \hat{C}(\ell_0, \delta) \subseteq \hat{C}(\ell, \delta))
 \end{aligned}$$

body analysed in the updated context δ_0 using the updated closure environment ce_0

where $\delta_0 = [\delta, \ell]_k$ and $ce'_0 = ce_0[x \mapsto \delta_0]$

x is now bound in the new calling context δ_0 but using the definition environment ce_0

Adding context: [app]

$$[app] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_0^{\ell_0}$$

$$(\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_m^{\ell_m}$$

$$(\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}, ce_0) \in \hat{C}(\ell_0, \delta) :$$

$$\forall \delta_b \in \hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta) :$$

$$\{\delta_b\} \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_{\delta_b}^{ce_0'} t_b^{\ell_b} \wedge$$

$$\hat{C}(\ell_b, \delta_b) \subseteq \hat{C}(\ell, \delta) \wedge \dots$$

where

$$ce_0' = ce_0[x_1 \mapsto \delta_b \downarrow_1, \dots, x_n \mapsto \delta_b \downarrow_n]$$

with $\downarrow_i$ the projection on the i-th component

The evolution of CFA for FUN

0-CFA

Context-insensitive and flow-insensitive flow analysis (merges all calls)

Polynomial-time higher-order analysis

Syntax-directed CFA

Analyze each body once at definition site, but analyzes unreachable code

Constraint-based formulation

Context-sensitive CFA (k-CFA style)

Track calling contexts and closure environments

Distinguishes different calls

Cartesian Product Algorithm

Analyze bodies for each argument tuple

Larger abstract state space

For all CFAs, semantics are preserved, and a (least) solution exists.

Moreover, an implementation can be provided

CFA for FUN: conclusion

Control-flow analysis for higher-order languages is a tradeoff between:

precision vs computational cost

More context $\Rightarrow$ fewer spurious flows $\Rightarrow$ higher complexity

The key challenge of CFA is balancing:

- soundness
- precision
- scalability